CO5	Experiment various clustering algorithms to solve real time problems
Task9:	Apply partitioning clustering algorithms for a given dataset. Compare the results and comment on the quality of clustering using evaluation metrics. **Platform: Google co-lab, Language: Python**

Use Case: Iris Flower Species Segmentation for Botanical Research

Background:
A botanical research institute aims to leverage partitioning clustering algorithms to analyze Iris flower species. The researchers have collected data on sepal length, sepal width, petal length, and petal width for a set of Iris flowers. The analysis will be conducted on Google Colab using Python.

Problem Statement:
The problem is to analyze the Iris dataset and apply different partitioning clustering algorithms to identify patterns or clusters among the data points representing different iris species. The goal is to compare the results of different partitioning clustering techniques, comment on the quality of clustering using evaluation metrics, and visually inspect the clusters using appropriate graphs.

Objective:
1. Apply different partitioning clustering algorithms to group iris data points based on their features.
2. Identify meaningful clusters within the Iris dataset representing different iris species.
3. Compare the results of different partitioning clustering algorithms.
4. Comment on the quality of clustering using evaluation metrics for each algorithm.
5. Visualize the clusters using appropriate graphs.

Algorithm: Iris Flower Species Segmentation using Partitioning Clustering

1. Import necessary libraries:
   - numpy, pandas, matplotlib.pyplot for data manipulation and visualization
   - KMeans, DBSCAN from sklearn.cluster for clustering algorithms
   - silhouette_score, davies_bouldin_score from sklearn.metrics for evaluation
   - load_iris from sklearn.datasets for loading the Iris dataset
   - StandardScaler from sklearn.preprocessing for data standardization
2. Define functions:

   2.1 Function load_and_preprocess_data():
      - Load the Iris dataset using load_iris()
      - Standardize the data using StandardScaler
      - Return the standardized data and labels

   2.2 Function apply_kmeans(data_std, n_clusters):
      - Apply K-Means clustering with the specified number of clusters (n_clusters)
      - Return the cluster labels

   2.3 Function visualize_kmeans(data_std, kmeans_labels):
      - Visualize K-Means clustering results using a scatter plot
      - Display the plot

2.4 Function evaluate_kmeans(data_std, kmeans_labels):
- Evaluate K-Means clustering using silhouette score and Davies-Bouldin index
- Print the evaluation metrics

2.5 Function apply_affinity (data_std, eps, min_samples):
- Apply Affinity clustering with specified epsilon (eps) and minimum samples (min_samples)
- Return the cluster labels

2.6 Function visualize_ affinity (data_std, dbscan_labels):
- Visualize DBSCAN clustering results using a scatter plot
- Display the plot

2.7 Function evaluate_ affinity (data_std, dbscan_labels):
- Evaluate DBSCAN clustering using silhouette score (excluding noise points)
- Print the evaluation metrics

2.8 Function plot_scatter(data_std, labels, title):
- Plot a scatter plot for clustering results
- Display the plot with specified title and axis labels

2.9 Follow Same for Birch Clustering

3. Main algorithm (in main()):
- Call load_and_preprocess_data() to obtain standardized data and labels

- Apply K-Means clustering:
  - Call apply_kmeans() with the standardized data and the desired number of clusters
  - Call visualize_kmeans() to visualize the clustering results
  - Call evaluate_kmeans() to evaluate the clustering performance

- Apply Affinity clustering:
  - Call apply_affinity() with the standardized data and specified epsilon and minimum samples
  - Call visualize_ affinity () to visualize the clustering results
  - Call evaluate_ affinity () to evaluate the clustering performance

- Apply Birch clustering:
  - Call apply_birch() with the standardized data and specified epsilon and minimum samples
  - Call visualize_ birch () to visualize the clustering results
  - Call evaluate_ birch () to evaluate the clustering performance

4. Execute the main algorithm:
- Call the main() function to perform the clustering analysis on the Iris dataset.

Program:

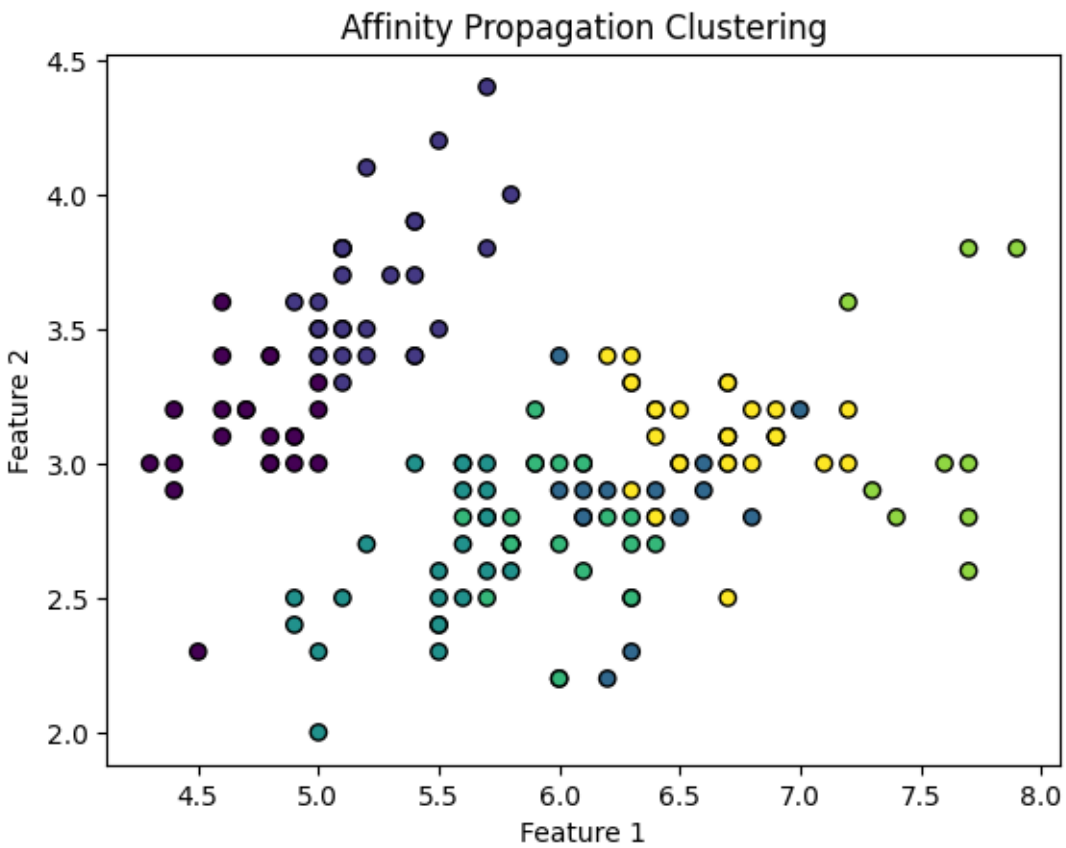
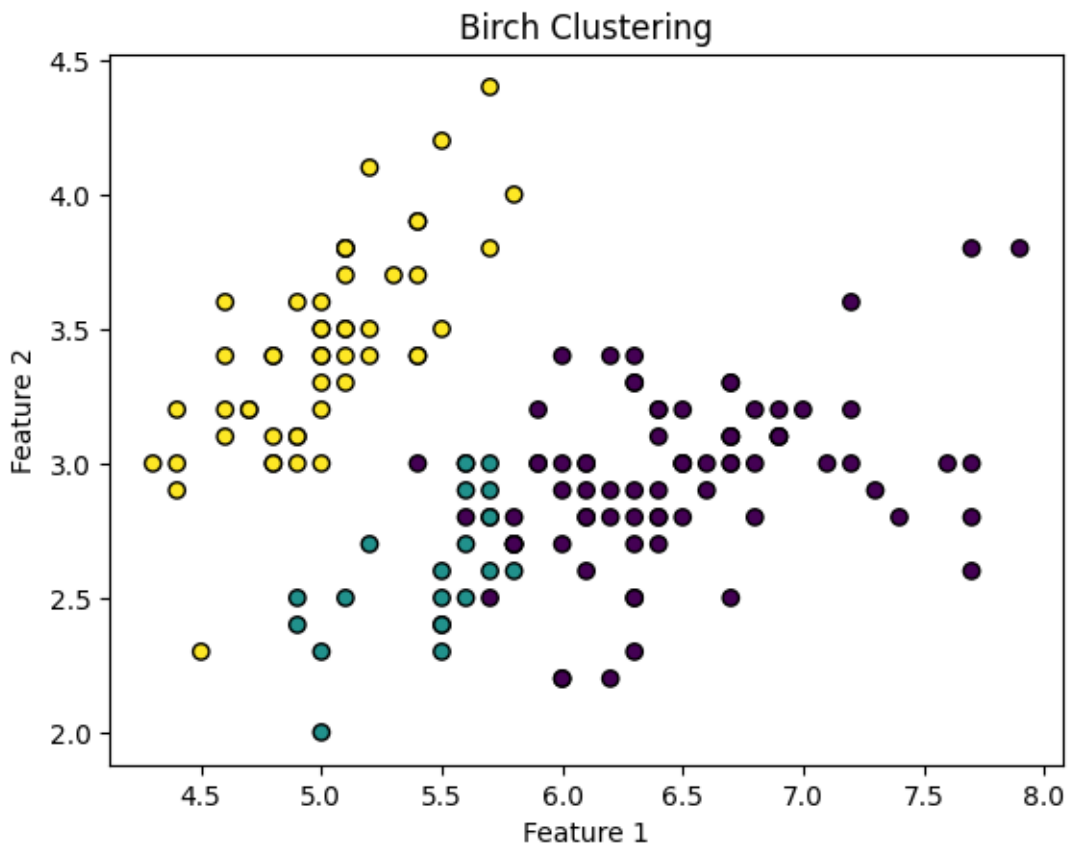
```
# Install required libraries
```

```
!pip install -q pandas numpy matplotlib scikit-learn

# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, AffinityPropagation, Birch
from sklearn.metrics import silhouette_score, davies_bouldin_score,
calinski_harabasz_score
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris()
data = pd.DataFrame(data= np.c_[iris['data'], iris['target']], columns=
iris['feature_names'] + ['target'])

# Select relevant features for clustering
selected_features = ['sepal length (cm)', 'sepal width (cm)', 'petal
length (cm)', 'petal width (cm)']
X = data[selected_features]

# K-means clustering function
def kmeans_clustering(X, n_clusters=3):
    model = KMeans(n_clusters=n_clusters, random_state=42)
    labels = model.fit_predict(X)
    return labels

# Affinity Propagation clustering function
def affinity_propagation_clustering(X):
    model = AffinityPropagation()
    labels = model.fit_predict(X)
    return labels

# Birch clustering function
def birch_clustering(X, n_clusters=3):
    model = Birch(n_clusters=n_clusters)
    labels = model.fit_predict(X)
    return labels

# Function to evaluate clustering metrics
def evaluate_clustering(X, labels, algorithm):
    silhouette = silhouette_score(X, labels)
    db_index = davies_bouldin_score(X, labels)
    ch_index = calinski_harabasz_score(X, labels)

    print(f'Evaluation Metrics for {algorithm}:')
    print(f'Silhouette Score: {silhouette}')
    print(f'Davies-Bouldin Index: {db_index}')
```

```
    print(f'Calinski-Harabasz Index: {ch_index}\n')

# Function to plot clusters
def plot_clusters(X, labels, algorithm):
    plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=labels, cmap='viridis',
marker='o', edgecolors='k')
    plt.title(f'{algorithm} Clustering')
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.show()

# Apply K-means clustering
kmeans_labels = kmeans_clustering(X)
evaluate_clustering(X, kmeans_labels, 'K-Means Clustering')
plot_clusters(X, kmeans_labels, 'K-Means')

# Apply Affinity Propagation clustering
affinity_labels = affinity_propagation_clustering(X)
evaluate_clustering(X, affinity_labels, 'Affinity Propagation')
plot_clusters(X, affinity_labels, 'Affinity Propagation')

# Apply Birch clustering
birch_labels = birch_clustering(X)
evaluate_clustering(X, birch_labels, 'Birch Clustering')
plot_clusters(X, birch_labels, 'Birch')
```

```
Evaluation Metrics for K-Means Clustering:
Silhouette Score: 0.5528190123564095
Davies-Bouldin Index: 0.6619715465007465
Calinski-Harabasz Index: 561.62775662962

/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870:
FutureWarning: The default value of `n_init` will change from 10 to 'auto'
in 1.4. Set the value of `n_init` explicitly to suppress the warning
  warnings.warn(
```

```
Evaluation Metrics for Affinity Propagation:
Silhouette Score: 0.3474081937055608
Davies-Bouldin Index: 0.9853972233056473
Calinski-Harabasz Index: 443.79711286686637
```



```
Evaluation Metrics for Birch Clustering:
Silhouette Score: 0.5019524848046077
Davies-Bouldin Index: 0.625830592433168
Calinski-Harabasz Index: 458.47251055625765
```